
Paged Flash-Decoding at Memory Speed: GQA on Tensor Cores, Fused Page Lookup, and FP8 KV Caches on the H100

Emaan Heidari Ara Esfarjani Kamsi Nwabueze

eheidari@usc.edu, aesfarja@usc.edu, knwabuez@usc.edu

Ming Hsieh Department of Electrical and Computer Engineering
University of Southern California, Los Angeles, CA, USA

Code: github.com/emaanh/paged-attention

Abstract

Decode attention streams the entire KV cache once per generated token, so its speed is set almost entirely by how close it runs to DRAM bandwidth. In an earlier FP32 study on an A40 we found that paging the KV cache cost a flat 1.79 times in kernel throughput, independent of page size. Here we build the kernel that removes that cost. It combines split-KV flash decoding with online softmax; grouped-query attention on tensor cores, where one `mma.sync` serves every query head in a KV group; page-table translation fused into each 16-byte `cp.async`; a persistent stream-K schedule that runs exactly one wave; and optional FP8 (E4M3) KV storage. On an H100 SXM it sustains 86% of peak DRAM bandwidth with an FP16 cache and 83% with FP8, measured by Nsight Compute. Against FlashInfer 0.7.0 on identical inputs, it is within 4% (median) with FP16 and 7% faster (median) with FP8. Sixteen-token pages now run within 0.8 points of an unpagged layout. An FP8 cache halves KV memory, doubling the sequences a fixed budget holds, and decodes 1.86 times faster than FP16 at a 3.9% relative output error.

1. Introduction

LLM inference alternates between a compute-bound *prefill* over the prompt and a memory-bound *decode* that generates one token at a time. Each decode step attends a single new query per head over every key and value cached so far:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \quad (1)$$

. Correspondence to: Emaan Heidari <eheidari@usc.edu>.

with $\mathbf{Q} \in \mathbb{R}^{G \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d}$, where G is the number of query heads sharing one KV head under grouped-query attention (Ainslie et al., 2023). Every element of $\mathbf{K}$ and $\mathbf{V}$ is read once and used in only $2G$ multiply-adds. At $G = 4$ with an FP16 cache that is 4 FLOP per byte, against an H100 ridge point near 295 FLOP per byte (NVIDIA, 2022). Decode is two orders of magnitude to the left of the ridge, so the only question that matters for a decode kernel is what fraction of DRAM bandwidth it sustains.

Serving systems add a second constraint: memory capacity. PagedAttention (Kwon et al., 2023) stores the cache in fixed-size pages mapped through a per-sequence block table, so a sequence holds only the pages it has filled rather than a region sized for the longest context allowed. That multiplies how many sequences fit, but it puts an indirection in the innermost loop of the most bandwidth-sensitive kernel in the system.

Where this work starts Our earlier study measured that indirection in isolation, on an A40, with an FP32 single-head flash-decode kernel shared between a contiguous and a paged pool. The paged pool was 1.79 times slower, and the gap did not move between 16 and 256 tokens per page. That localized the cost to address translation on every element access, rather than to page-boundary crossings. That paper ended with a list of fixes: keep block tables resident on the device, resolve a page once rather than per element, decode every sequence in one launch, and test against a fragmented pool. This paper carries out that list on a modern GPU and a realistic model shape, and adds the two features production decode kernels need, GQA and a quantized cache.

1.1. Contributions

1. **A paged split-KV decode kernel on tensor cores.** GQA groups map onto the rows of `mma.sync.m16n8k16`. A k-slot permutation and an XOR swizzle make every shared-memory fragment

Table 1. Summary on one H100 SXM, Llama-3-8B layer shape ($H_q=32$, $H_{kv}=8$, $d=128$), 16-token pages scattered through the pool.

Measurement	FP16 KV	FP8 KV
DRAM throughput, Nsight Compute	86.3%	82.6%
Bandwidth, long-context shapes	83–88%	74–83%
Latency vs. FlashInfer (median)	+4.0%	−6.2%
Paging cost (16-token pages vs. none)	0.8 pts	+0.2 pts
Decode time, $32 \times 8K$	364 μs	196 μs
Sequences in 1 GiB, 2K avg., 8K max	128	256

A40 FP32 predecessor: 73% of bandwidth; paging cost 1.79×

load a conflict-free 16-byte access, for both FP16 and FP8 storage (Section 3.4).

2. **Fused page lookup and a persistent schedule.** Each 16-byte `cp.async` translates its own page. A host-built stream-K plan (Osama et al., 2023) gives every CTA an equal, contiguous run of tiles in a single wave, and the pipeline streams across sequence boundaries (Sections 3.3–3.6).
3. **An FP8 KV cache** quantized on append and dequantized in registers straight into MMA fragments (Section 3.5).
4. **A measured account** on an H100: bandwidth across 14 shapes, Nsight Compute counters, a head-to-head against FlashInfer (Ye et al., 2025) on identical tensors, page-size and grid-size sweeps, allocator-measured capacity, and FP8 accuracy. The optimization steps that got there are also reported, including the ones that did not work (Section 4).

2. Background and Related Work

Flash decoding and online softmax FlashAttention (Dao et al., 2022; Dao, 2024) tiles the query matrix for prefill. At decode the query is a single row per head, so the only parallelism is along T . Flash-Decoding (Dao et al., 2023) splits T across thread blocks and merges their partials. This is possible because the online softmax of Milakov & Gimelshein (2018) makes a partial (o, m, ℓ) over any subset of tokens composable by log-sum-exp.

PagedAttention vLLM (Kwon et al., 2023) introduced paged KV caches for serving. It evaluates them at the system level on request traces. Our focus is the kernel-level cost of the indirection and how to remove it.

FlashInfer FlashInfer (Ye et al., 2025) is a production attention library for serving, with paged decode kernels on CUDA cores and on tensor cores, FP8 KV support, and a load-balanced plan/run split. It is our baseline. Our kernel reads FlashInfer’s HND paged layout directly, so both run on the same buffers.

Grouped-query attention GQA (Ainslie et al., 2023) shares each KV head among G query heads, cutting KV size by G at little quality cost. For decode it also raises arithmetic intensity by G . That makes tensor cores worthwhile once G query rows can be packed into one MMA.

Stream-K and FP8 Stream-K (Osama et al., 2023) splits GEMM work evenly across a persistent grid to remove wave quantization. We apply the same idea to decode tiles. FP8 E4M3 (Micikevicius et al., 2022) keeps 4 exponent and 3 mantissa bits. Hopper converts pairs of E4M3 values to FP16 in hardware.

3. Design

3.1. Cache layout and allocator

K and V each live in one pool of shape $[pages][H_{kv}][P][d]$ (the HND layout), so one head’s slice of one page is a contiguous run of P rows. A free list hands out pages. Each sequence owns a row of a block table that is resident on the GPU, and only newly allocated entries are copied, when a sequence grows onto a new page. Decode reads nothing from the host. `append` scatters new tokens into their pages and, for an FP8 cache, quantizes them with per-tensor scales s_k, s_v (stored value = x/s).

3.2. Kernel structure

A CTA of four warps streams tiles of 64 tokens through a double-buffered `cp.async` pipeline in shared memory. Each warp owns 16 tokens of every tile and keeps its own online-softmax state: running max m , running sum ℓ , and an FP32 output accumulator. The only CTA-wide barriers are therefore the pipeline’s two per tile. When a CTA finishes a segment (Section 3.6), its four warps merge through shared memory and write either the final output, if the segment covered the whole sequence, or a partial $(o, \log_2 \ell + m)$ for a separate merge kernel.

3.3. Fused page lookup

Each thread issues 16-byte `cp.async` copies. The source address of each copy is formed from the token’s physical page, read from the block table through the read-only cache:

```
page = __ldg(bt + token / page_size);
src = page*page_stride + kv_head*head_stride
      + (token % page_size)*row_bytes + chunk*16;
cp_async_16(dst, kv_base + src, token < end);
```

There is one lookup per 16 bytes, not per element. The lookup also sits in the producer, which runs a full tile ahead of the math, so its latency overlaps with the loads already in flight. Tokens past the end of a segment are zero-filled (source size 0), which keeps masked rows finite in the **PV** product.

3.4. GQA on tensor cores

Scores $S = QK^T$ for 16 tokens is two m16n8k16 MMAs per k-step, and $d=128$ gives 8 k-steps. The $G \leq 8$ query heads fill rows 0–7 of the A operand, and rows 8–15 are zero. That wastes half of each MMA, which costs nothing here, because the kernel is bound by bandwidth rather than tensor-core throughput. Q is loaded into registers once per segment.

Conflict-free fragments The B-fragment layout of m16n8k16 gives lane (r, q) the k-slots $\{2q, 2q+1, 2q+8, 2q+9\}$ of column r . Loaded literally, a lane’s 4 elements are scattered across the row. The dot product does not care which head dimension sits in which k-slot, as long as Q uses the same assignment. So we assign each lane’s k-slots in step k to 4 *consecutive* dimensions, chosen so that a lane’s dimensions across all 8 steps form 16-byte chunks: 4 chunks per row in FP16, 2 in FP8. Rows are stored with their 16-byte chunk index XOR-swizzled by bits 0, 2 and 3 of the token row. With this swizzle, both the QK access (8 rows by 4 chunk columns) and the PV access (4 rows by 8 chunk columns) hit 8 distinct bank groups per quarter-warp. Nsight counts 18,221 conflicts in a 1 GB FP16 decode, which is effectively none.

Weighted sum The token-to-column mapping of the score MMAs is chosen so that lane quad q ends up holding the scores of tokens $4q..4q+3$. Those are exactly the tokens its PV A-fragment needs, so P goes from score registers to MMA operand with one FP32→FP16 pack and no shuffles. For the V B-fragments, a lane reads 4 token rows at 16 consecutive dimensions, and pairs of tokens are interleaved with `_byte_perm`. Output column j of n-tile t is then dimension $16j + t$, which the epilogue undoes when it writes the result.

3.5. FP8 storage

FP8 rows are 128 bytes, so each 64-token tile moves half the bytes. The same fragment code runs. The K words feed `_nv_cvt_fp8x2_to_halfraw2` directly, since each 16-bit half of a word holds two consecutive dimensions. V bytes are first interleaved across token pairs with one `_byte_perm` per pair, then widened. s_k is folded into the softmax scale, and s_v into the final normalization.

3.6. Persistent stream-K schedule

The work is the ordered list of (sequence, KV head, tile) triples. A host-side plan cuts it into C equal contiguous ranges, where C is one wave of CTAs. Each maximal run of one (sequence, head) inside a range is a *segment*. Sequences covered by a single segment are written directly, and the rest go through a log-sum-exp merge kernel. Inside the

Table 2. Percent of peak DRAM bandwidth at batch 32, 8K context, after each step.

Step	FP16	FP8
CUDA cores, 16 KB tiles, 3 stages, split grid	66.3	42.7
+ smaller tiles, 4 CTAs/SM	80.8	52.0
Tensor-core GQA (split grid)	83.2	68.0
+ persistent stream-K grid	87.9	81.8

kernel, the producer walks the CTA’s segments one tile ahead of the consumer, so the pipeline never drains at a segment boundary. As in FlashInfer, the plan depends only on sequence lengths. It is built once per decode step and reused by every layer, so it is not in the timed region.

4. How We Got There

Table 2 records bandwidth at $32 \times 8K$ after each design change. All numbers come from the same benchmark on the same GPU.

CUDA cores first The first version computed dot products on CUDA cores, with 16 threads per token and shuffle reductions. Nsight put it at 12.5% occupancy, limited by shared memory, issuing 0.44 instructions per scheduler cycle: latency-bound, not bandwidth-bound. Halving the tile and quadrupling resident CTAs brought FP16 to 81%. FP8 stayed near 50%. It packs twice the tokens into each byte, so it needs twice the instructions per byte, and the kernel hit the issue limit before it hit bandwidth.

Tensor cores Moving both products onto `mma.sync` cut the instruction count per byte to about a third. FP8 jumped to 68–77% and FP16 to 80–84% across shapes.

Wave quantization With a split grid, bandwidth swung by up to 30 points between shapes and split counts. That happened whenever the CTA count was just past a multiple of the resident capacity (e.g. 512 CTAs on 396 slots), and also whenever CTAs were short, since each one pays a full DRAM round trip before it streams. A wave-aware split-count heuristic helped on some shapes and hurt on others. The persistent schedule fixed both problems at once. After it, every long-context shape falls within 83–88% (FP16).

What did not help Letting FP16 run 3 CTAs per SM, by moving the warp merge into the just-consumed pipeline stage, gained 3 points at batch 64 and lost 4–12 points elsewhere. The default grid is therefore 2 CTAs/SM for FP16 and 3 for FP8 (Figure 3). Bigger CTAs (8 warps) and deeper pipelines (3–4 stages) were within noise or worse.

Table 3. Platform.

GPU	H100 SXM, 80 GB HBM3, 132 SMs
Peak DRAM bandwidth	3,352 GB/s (theoretical)
Streaming-read ceiling	93.1% of peak
Host	Modal cloud container
Toolchain	CUDA 12.8, C++20 host / C++17 device
Baseline	FlashInfer 0.7.0, PyTorch 2.8
Profiler	Nsight Compute 2025.1
Correctness	GoogleTest
Shape	$H_q=32, H_{kv}=8, d=128, P=16$
KV dtypes	FP16; FP8 E4M3, per-tensor scale

5. Experimental Setup

Timing Each configuration is planned once. Then, for every iteration, a buffer twice the L2 size is streamed to evict the cache, and only the decode kernels are timed with CUDA events. We report the median of 50 iterations. FlashInfer runs through the same harness on the same tensors and block table, with both its CUDA-core and tensor-core decode paths tried; we keep the faster. Head-to-head numbers are the median of 3 repeats. Separate runs on the same GPU differ by about 2–3%. Clocks were not locked, because the container could not lock them.

Bandwidth Effective bandwidth counts every byte the kernel must read (all of **K** and **V**, plus **q**, the output, and the block table) over GPU time, as a percentage of the theoretical 3,352 GB/s. Nsight Compute’s `dram_throughput` is reported separately.

Fragmentation Pages are drawn from a shuffled free list, so each sequence’s pages are scattered through the pool, the adversarial case our earlier study could not test.

Correctness Every configuration is checked against a double-precision CPU reference on the exact stored values: FP16 and FP8; $G \in \{1, 2, 4, 8\}$; page sizes 1–128; mixed-length batches including 1-token sequences; lengths off the page and tile grid; interleaved appends; 32K tokens; and grids of 1 to 1,000 CTAs. The maximum error is below 4×10^{-3} , the FP16 output rounding. Our outputs match FlashInfer to 2.4×10^{-4} and an FP32 PyTorch reference to 1.2×10^{-4} .

6. Results

6.1. Bandwidth

Figure 1 shows bandwidth across 14 shapes. With at least 4K tokens per sequence and at least 100 MB of KV, FP16 holds 83–88% and FP8 74–83%. The best FP16 case is 2.95 TB/s, 94% of what a bare streaming read attains. Short problems are latency-bound: a $1 \times 4K$ decode moves 17 MB in about 20 μ s, a time comparable to the launch and pipeline fill. Batches of 1K-token sequences are cut into more segments,

Table 4. Nsight Compute counters for the decode kernel.

	Shape	DRAM %	Read/needed	Warps active %
FP16	$32 \times 8K$	86.1	1.001	12.3
FP16	$8 \times 32K$	86.3	1.001	12.3
FP16	$128 \times 2K$	82.6	1.001	12.2
FP16	$1 \times 128K$	83.5	1.000	12.2
FP8	$32 \times 8K$	81.6	1.001	18.3
FP8	$8 \times 32K$	82.6	1.001	18.3
FP8	$128 \times 2K$	78.3	1.003	18.2
FP8	$1 \times 128K$	77.4	1.000	18.0

Table 5. Decode latency in μ s, this kernel against FlashInfer 0.7.0 (faster of its two paths). Ratio = FlashInfer / ours; above 1 means ours is faster.

Shape	FP16 KV			FP8 KV		
	Ours	FI	Ratio	Ours	FI	Ratio
$1 \times 16K$	39.7	36.2	0.911	31.6	33.0	1.044
$1 \times 64K$	106.3	103.6	0.974	65.4	69.8	1.067
$1 \times 128K$	196.4	192.4	0.980	110.5	119.2	1.079
$8 \times 4K$	60.4	57.5	0.953	42.2	47.0	1.113
$8 \times 16K$	199.2	192.1	0.964	109.3	120.8	1.105
$8 \times 32K$	377.5	366.9	0.972	200.3	213.4	1.065
$32 \times 1K$	56.4	56.3	0.999	42.8	44.7	1.044
$32 \times 4K$	196.9	188.9	0.959	109.8	118.3	1.077
$32 \times 8K$	375.5	364.7	0.971	200.6	212.7	1.060
$64 \times 4K$	393.3	365.2	0.929	203.0	214.9	1.059
$128 \times 1K$	204.2	188.3	0.922	115.8	129.4	1.117
$128 \times 2K$	391.1	365.9	0.935	207.1	219.5	1.060

and the per-segment merge becomes visible.

Table 4 gives the Nsight Compute view. DRAM bytes read match the bytes the kernel needs to within 0.1%, so nothing is fetched twice, and `dram_throughput` peaks at 86.3% (FP16) and 82.6% (FP8) of peak.

6.2. Against FlashInfer

Table 5 compares latency on identical inputs. With an FP16 cache FlashInfer is faster, by 4.0% at the median and at most 9.7%. The largest gaps are at batch 64–128 and at $1 \times 16K$. With FP8 our kernel is faster on every shape: FlashInfer takes 4–12% longer (median 6.6%). The dequantization runs inside the same tensor-core inner loop as FP16 and adds only two to three instructions per fragment. FlashInfer’s faster variant was its tensor-core path on every FP8 shape.

6.3. The cost of paging

Figure 2 sweeps the page size. The 8,192-token point is one page per sequence, so it is the contiguous layout read through the same kernel. From 16 tokens up, FP16 stays within 0.8 points of it, and FP8 is flat within noise. On the A40 the same comparison cost 1.79 times. Below 8 tokens per page, a 64-token tile touches 8 or more pages, the lookups stop amortizing, and bandwidth falls to 54% at one token per page.

The answer to our earlier paper’s question is therefore that paging costs almost nothing, once the lookup is paid per

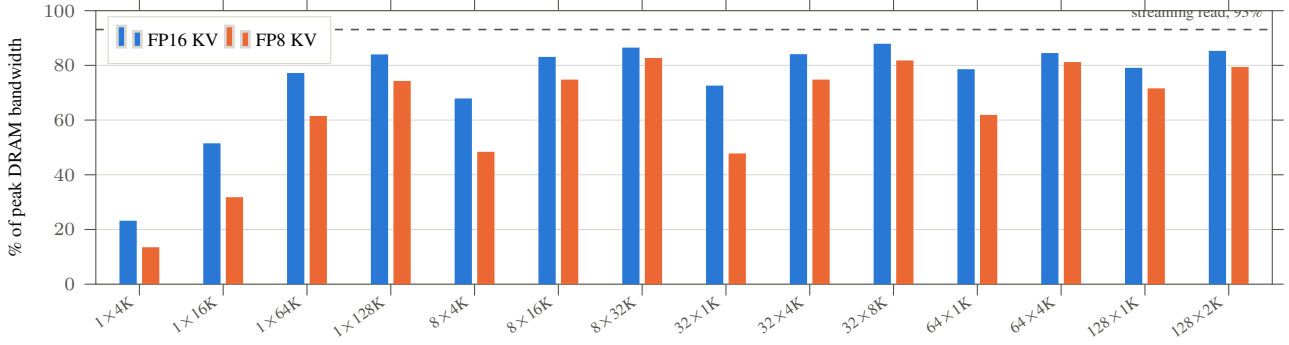


Figure 1. Effective bandwidth across batch $\times$ context length. The dashed line is a plain 16-byte streaming read, the practical ceiling.

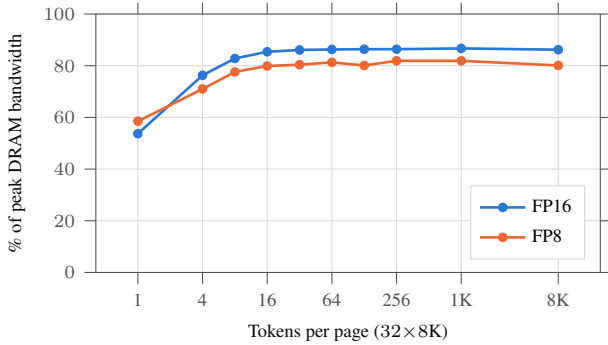


Figure 2. Bandwidth against page size. The 8K point is one page per sequence, i.e. an unpaged layout.

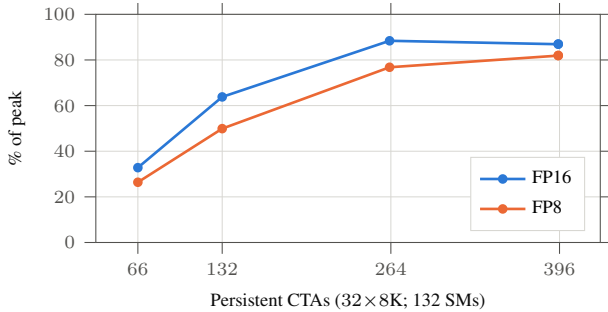


Figure 3. Bandwidth against persistent grid size. FP16 peaks at 2 CTAs per SM, FP8 at 3.

16-byte load, is hidden behind the pipeline, and reads a device-resident table. The 1.79 times we measured before was a property of that implementation, not of paging.

6.4. Grid size

With one CTA per SM, four warps cannot keep enough bytes in flight (Figure 3). FP16 peaks at two CTAs per SM and FP8 at three. FP8 does twice the math per byte, so it benefits from more warps to hide latency.

Table 6. Sequences that fit 1 GiB of one layer’s KV, with an 8K context limit. Paged counts come from running the allocator to exhaustion.

Tokens used	Contig. FP16	Paged FP16	Paged FP8	FP8 / contig.
256	32	1024	2048	64 $\times$
512	32	512	1024	32 $\times$
1024	32	256	512	16 $\times$
2048	32	128	256	8 $\times$
4096	32	64	128	4 $\times$
8192	32	32	64	2 $\times$

6.5. Capacity

A contiguous cache reserves the full 8K context per sequence. Paging allocates per 16 tokens. FP8 halves every page. Table 6 gives the counts: paged FP16 fits 8192/len times the contiguous count, and paged FP8 exactly twice paged FP16 at every length. The gain over contiguous depends entirely on how far below the context limit real sequences run. The factor of two from FP8 does not.

6.6. FP8 accuracy and speed

Against the same decode over the unquantized FP16 cache, FP8 outputs have a relative ℓ_2 error of 3.8–3.9% on every shape. That is consistent with E4M3’s 3 mantissa bits and a single per-tensor scale. Finer (per-head or per-token) scales would reduce it without changing the kernel’s inner loop. In exchange, FP8 decodes 32 $\times$ 8K in 196 μ s against 364 μ s, 1.86 times faster, or 93% of the ideal 2 times.

7. Discussion

Why FP8 trails FP16 in bandwidth but not in time Per byte, FP8 carries twice the tokens, so twice the MMAs and softmax work, plus the widening. It runs a few points further from the DRAM roof. It still reads half the bytes, so it finishes in 54% of the FP16 time.

Where FlashInfer is ahead FP16 at batch 64–128 is where a CTA crosses the most sequence boundaries (about

5 per CTA at $128 \times 2K$). Each crossing costs a warp merge through shared memory, a barrier, and a load of the next sequence’s Q that is not prefetched. This is our best explanation for the remaining 7–8%, but we have not profiled it in isolation. Prefetching Q with the first tile of each segment is the obvious next step.

Threats to validity One GPU, one model shape, and $d=128$ only. Clocks were unlocked on a shared cloud host, and separate runs vary by about 2–3%, which is comparable to some of the differences we report. The head-to-head is still paired, with both kernels timed back to back on the same inputs. FlashInfer was run through its public API with default settings; a tuned configuration might close or widen the gaps. The capacity study uses fixed sequence lengths rather than a request trace.

8. Conclusion

A paged decode kernel does not have to pay for paging. Translating pages per 16-byte load inside an asynchronous pipeline, running GQA groups through tensor cores, and scheduling a single persistent wave of equal work together sustain 86% of H100 peak DRAM bandwidth with an FP16 cache, within a few percent of FlashInfer. With an FP8 cache the kernel is faster than FlashInfer on every shape we measured, holds twice the sequences, and decodes 1.86 times faster than FP16. Relative to our A40 predecessor, the 1.79 times paging penalty is gone. What remains between the kernel and the streaming-read ceiling does not depend on the memory layout; our best candidate is per-segment overhead.

Artifact Availability

The kernel, allocator, tests, benchmarks, the FlashInfer harness, and the CSVs behind every table and figure are at github.com/emaanh/paged-attention. This paper’s tables and plots are generated directly from those CSVs by a script that accompanies the paper source.

References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Dao, T., Haziza, D., Massa, F., and Sizov, G. Flash-decoding for long-context inference. Stanford CRFM, 2023. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
- Milakov, M. and Gimelshein, N. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- NVIDIA. NVIDIA H100 Tensor Core GPU architecture. Whitepaper, 2022.
- Osama, M., Merrill, D., Cecka, C., Garland, M., and Owens, J. D. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In *Proceedings of PPoPP*, 2023.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.